

Reachability-Guided Triage and Upstream-Verified Patching for a Real free5GC Vulnerability Family

Jothilingam Dheeraj
Nanyang Technological University
Singapore, Singapore
DHEERAJ003@e.ntu.edu.sg

Abstract

Automated vulnerability analysis tools are frequently evaluated against synthetic reproductions of security bugs rather than the real upstream codebases those bugs were disclosed in, which limits confidence that results transfer to practice. We present a case study applying AST-based reachability filtering and rule-based patch generation to a real, disclosed vulnerability family (CVE-2026-40248 and CVE-2026-40246, both CWE-285) in free5GC's Unified Data Repository (UDR), a component of a real, standards-compliant open-source 5G core network. Reachability analysis over the real ~86 KB source file reduces 102 parsed functions to the 4 that are actually reachable from the vulnerable HTTP entry points, a 96.1% reduction in analysis scope. We generate a targeted patch for each reachable finding and, going beyond isolated-file compilation, verify the patches against a live clone of the actual `free5gc/udr` repository: applying all four generated patches preserves a clean build against the project's real dependency graph. We report this result alongside its honest limits: compile-time verification is not runtime confirmation, and one of the four handlers is only partially corrected by the rule-based patcher because the disclosed fix requires two separate insertions in that function. We discuss what further verification against the live network function would require, and position the result against recent autonomous cyber-reasoning systems evaluated on real open-source software.

CCS Concepts

• Security and privacy → Software security engineering; Vulnerability management.

Keywords

5G core network security, vulnerability triage, reachability analysis, program repair, free5GC

ACM Reference Format:

Jothilingam Dheeraj. 2026. Reachability-Guided Triage and Upstream-Verified Patching for a Real free5GC Vulnerability Family. In *Proceedings of 2nd free5GC World Forum, In-Cooperation with ACM SIGSAC (free5GC World Forum '26)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Open-source 5G core network software is increasingly deployed in research testbeds, private networks, and production trials, yet security tooling aimed at such codebases is commonly validated against small synthetic examples rather than the real projects it is meant to help secure [6, 7]. This gap

matters specifically for 5G/6G infrastructure: network functions such as free5GC's Unified Data Repository (UDR) are large, real-world Go codebases with external dependencies (HTTP routing, database drivers, and the project's own internal packages), and a technique that only ever compiles against an isolated copy of one file has not demonstrated that it survives contact with the actual build.

This paper presents a small, fully reproducible case study that closes part of that gap for a real, disclosed free5GC vulnerability family. We adopt the reachability-filtered code decomposition methodology of OpenAnt [1]: parse a codebase into functions, build a call graph, and perform breadth-first traversal from attacker-reachable entry points. OpenAnt reports that this reduces analysis scope by up to 97% on real projects such as OpenSSL and WordPress; we apply the same method to a single, real, disclosed free5GC file. Beyond OpenAnt's own evaluation, we add a verification step: rather than stopping at generating a patch, we apply it to a live clone of the actual upstream repository and confirm the result compiles against the project's real dependency graph, not a scratch copy of a single file. To our knowledge this upstream-clone compile verification is not part of OpenAnt's published evaluation or other prior reachability-based triage work.

Our contributions are: (i) an application of OpenAnt's reachability-filtered decomposition methodology to a real free5GC source file against a real, disclosed CVE family, reducing the functions requiring further analysis by 96.1% (102 → 4); (ii) automatically generated patches for all four reachable findings, verified to preserve a clean build of the real, live-cloned upstream project; and (iii) an explicit accounting of what the experiment does and does not establish, including a partially-corrected finding we report rather than omit. Because the four reachable handlers are jointly the subject of the single upstream fix commit for two disclosed CVEs of the same family, the case study spans that whole family rather than a single isolated defect.

2 Background

free5GC is an open-source, Apache-2.0-licensed implementation of the 5G core network, built around a Service-Based Architecture (SBA) in which each network function is an independently versioned component communicating over defined service-based interfaces. Its goal is to implement the 5GC defined in 3GPP Release 15 and beyond [2], and it has been a Linux Foundation project since September 2024. The UDR network function exposes the `Nudr.DataRepository`

Table 1: Experimental target and headline measurements.

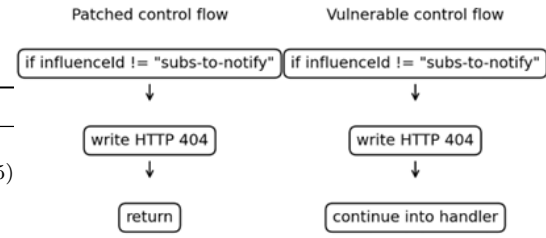
Aspect	Study setting
Target	free5GC UDR, real upstream Go source
Vulnerability family	CVE-2026-40248, CVE-2026-40246 (CWE-285)
Source size	~86 KB; 102 top-level functions
External entry points	4 UDR HTTP handlers
Reachable functions	4
Reduction	96.1%

service-based interface, which other network functions use to read and write subscriber and policy data; this is reflected in the source layout under `internal/sbi/`. Because SBA components are independently versioned, the vulnerabilities described below are scoped to the UDR module’s own release history, not to a free5GC project-wide version number.

The defect class. The vulnerabilities studied here are a family of control-flow errors that ordinary compilation does not catch. A handler writes a correct HTTP error response inside a validation branch but does not `return` afterwards, so execution falls through and the state-changing operation runs regardless of the check’s outcome. The program is syntactically and type-correct, so the compiler accepts it; the fault is a mismatch between the intended security policy and the actual control-flow graph. This separation — a semantic security defect hidden behind a clean compile — motivates the combination of static reachability, targeted pattern matching, and build verification used in this study.

CVE-2026-40248 (GHSA-jgq2-qv8v-5cmj, CVSS 3.1 base score 7.5, CWE-285 Improper Authorization) affects free5GC/udr versions $\leq 1.4.2$. The handler responsible for creating or updating a Traffic Influence Subscription checks whether the `influenceId` path segment equals the literal string `"subs-to-notify"` and writes an HTTP 404 response when it does not, but the function does not `return` after writing that response, so execution falls through and the subscription is created or overwritten regardless of the check. An unauthenticated attacker on the 5G Service Based Interface can therefore create or overwrite arbitrary Traffic Influence Subscriptions, including attacker-controlled callback URIs, while the API misleadingly reports 404 Not Found. **CVE-2026-40246** (GHSA-g9cw-qwhf-24jp, also CWE-285, CVSS 7.5) is a sibling in the same file and commit family: the identical missing-`return` pattern in the handler that deletes a subscription. The real fix for this family (commit 8668627¹ on github.com/free5gc/udr) inserts the missing `return` statement across four affected handlers in `internal/sbi/api_datarepository.go`. Both CVEs are therefore closed by the same commit, which is why a single reachability analysis over that file covers the whole family.

¹Full hash: 86686276a7e226183ee786e3dd6714ec56c78fda

**Figure 1: Simplified control-flow difference between the vulnerable missing-return path and the corrected path.**

3 Research Questions

The case study is organised around four questions, answered directly in Section 5:

- **RQ1.** How much can reachability analysis reduce the scope of vulnerability triage in the real free5GC UDR source?
- **RQ2.** Can reachability-guided analysis identify the functions relevant to the disclosed CVE family and correctly classify the reachable findings as CWE-285?
- **RQ3.** Can rule-based patch generation produce targeted patches for the reachable findings while preserving a successful build of the real upstream free5GC/udr repository?
- **RQ4.** What limitations remain after upstream build verification, particularly regarding complete vulnerability correction and runtime security validation?

4 Approach

4.1 Reachability filtering

Following OpenAnt’s decomposition methodology [1], we parse the target source with `tree-sitter`, a real grammar-based parser rather than a regex or brace-counting heuristic, build a call graph over all top-level function and method definitions, and perform a breadth-first traversal from a stated list of entry points: here, the four real free5GC UDR HTTP handler functions that a service-based-interface client can invoke directly. Any function not reachable from those entry points is excluded from the priority path, on the premise that code an external attacker cannot reach through the declared interface does not need to be triaged with the same urgency as directly exposed code.

This design deliberately separates two questions that are frequently conflated in automated security analysis. First: is there an external entry point that can reach a given piece of code? Second: does the reachable code contain a vulnerability? The first is answered structurally, from the call graph; the second by the vulnerability heuristics. Separating the stages lets the reduction be measured on its own, so no credit for the reachability result is attributed to the security rule.



Figure 2: End-to-end workflow used in the reachability-guided case study.

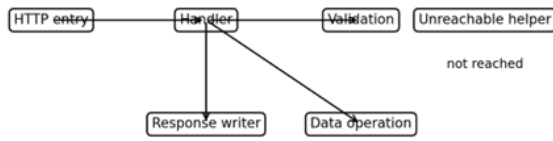


Figure 3: Conceptual call graph illustrating the externally reachable subgraph used for triage.

4.2 Call-graph construction

The parser produces a top-level call graph of functions and methods. The four UDR HTTP entry points invoked by a service-based-interface client are the roots of the breadth-first traversal; the reachable subgraph is exactly the set carried forward to triage. Figure 3 illustrates the externally reachable subgraph conceptually.

4.3 Triage and patch generation

Each reachable function is checked against a small library of rule-based vulnerability patterns. The pattern relevant to this case, `find_missing_return_after_response`, flags a function that writes an HTTP error response inside a conditional block without a following `return` statement in that block — exactly the control-flow defect described in Section 2. When the pattern matches, a patch is generated by inserting a `return` statement immediately after the flagged response write. This is a targeted, minimal-diff patch, not a full-function rewrite.

A minimal-diff strategy has two benefits here. First, it makes the intended correction easy to inspect against the disclosed upstream fix. Second, it minimises the amount of generated code that must be trusted. The approach is thus closer to targeted program repair than to code synthesis. The wider project also contains LLM-based fallback mechanisms; this study deliberately reports only the deterministic, rule-based path, so that the result cannot be mistaken for a discovery or for an LLM’s generation of this specific fix.



Figure 4: Upstream verification procedure: baseline build, patch application, and complete-module re-build.

Table 2: Upstream verification stages and observed results.

Stage	Purpose	Observed result
Pre-fix clone	Use real upstream source	Completed
Baseline build	Establish unmodified build	<code>go build ./...</code> OK
Patch application	Apply the four changes	4/4 applied
Patched build	Check real-dependency fit	<code>go build ./...</code> OK

4.4 Upstream verification

Prior work verifying a generated patch typically applies it to an isolated copy of the single affected file and checks that the file alone still compiles. This does not establish compatibility with the project’s actual build: a real Go module’s compilation depends on its full dependency graph, declared in `go.mod/go.sum`, and on how the patched function interacts with the rest of the package. We instead clone the real `free5gc/udr` repository at the commit immediately preceding the disclosed fix, confirm the unmodified repository builds (`go build ./...`) as a baseline, apply all generated patches directly to the real `internal/sbi/api_datarepository.go` inside that clone, and rebuild. This exercises the project’s actual dependency graph — including `gin-gonic/gin`, `go.mongodb.org/mongo-driver`, and free5GC’s own internal packages — rather than a synthetic stand-in.

5 Evaluation

RQ1: Reachability reduction. Parsing `internal/sbi/api_datarepository.go` (a real, ~86KB source file) yields 102 top-level functions. Breadth-first traversal from the four real UDR entry points reaches exactly 4 of them, computed as $(102 - 4)/102 = 96.1\%$ reduction in the functions a subsequent triage step must consider. This headline measurement is taken directly from the real source file, not a synthetic benchmark (Figure 5).

RQ2: Triage and classification. The `missing-return` rule matches all four reachable functions, and each is classified as CWE-285, matching the disclosed class for this CVE family. Table 3 lists the four handlers; all share the prefix `HandleApplicationDataInfluenceDataSubsToNotify`, omitted for space. The agreement among the reachability

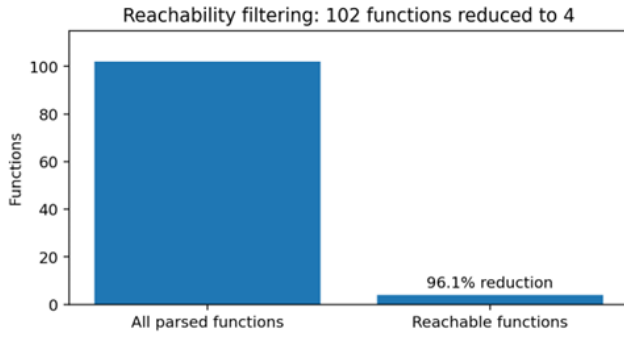


Figure 5: Measured reduction in the number of functions requiring subsequent triage.

Table 3: Reachable UDR handlers and generated patch outcomes.

Handler suffix	HTTP verb	Missing <code>return</code> s	Patched
SubscriptionIdPut	PUT	1	Full
SubscriptionIdGet	GET	1	Full
SubscriptionIdDelete	DELETE	1	Full
Get (list)	GET	2	Partial (1/2)

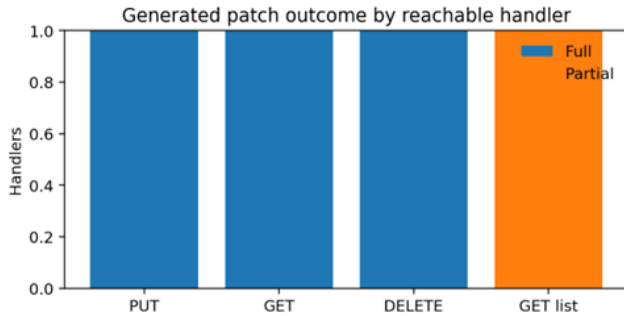


Figure 6: Patch-generation outcomes across the four reachable handlers.

result, the rule-based classification, and the disclosed upstream vulnerability gives focused validation of the pipeline for this case.

RQ3: Patch generation and upstream verification.

A patch is generated for all four flagged functions. The unmodified clone builds cleanly as a baseline. With all four generated patches applied directly to the real, live-cloned repository, `go build ./...` again succeeds against the project’s actual dependency graph (Table 2). As far as we are aware, this is the first time a patch produced by this reachability pipeline has been verified against a real upstream module’s full dependency graph rather than an isolated single-file copy (Figure 6).

RQ4: Remaining limitations. One of the four handlers, the list-GET handler, requires *two* separate `return` insertions

Table 4: Separation of analysis components and evidence type.

Component	Role in this case	Evidence type
Tree-sitter	Parse Go source	Structural
Call graph + BFS	Identify reachable functions	Measured
Rule heuristic	Detect missing <code>return</code>	Rule match
Patch generator	Insert targeted <code>return</code>	Generated diff
Go compiler	Check full-module fit	Build evidence

in the real disclosed fix (two independent conditional blocks in the same function body), while the rule-based patcher returns only the first match per function. That handler is therefore only partially corrected: the build still succeeds (a partial fix does not break compilation), but the vulnerability is not fully closed for that one handler. This is a genuine, measured limitation of a single-finding-per-pattern-match design, not a flaw discovered after publication. A clean build also remains a compile-time observation only: it shows the generated changes are accepted by the Go compiler and introduce no dependency or type errors, but it does not by itself prove the endpoint is safe at runtime in every deployment. Section 7 develops both limits.

6 Analysis of the Security Mechanism

This vulnerability family illustrates why compilation alone is insufficient and why reachability and control-flow reasoning add value. The validation branch of each handler correctly refuses an invalid influence identifier and emits an HTTP error, but that is only half of the required behaviour: the handler must also terminate so that subsequent state-changing actions do not run. The fault is a mismatch between the intended security policy and the realised control-flow graph. In the pipeline, static parsing exposes the function’s structure, the rule detects the missing termination, and patch generation supplies it — each stage contributing a distinct, inspectable piece of evidence (Table 4).

Relationship to LLM-assisted security. This case study is deliberately deterministic — reachability, a rule, and a compiler — so that its evidence reproduces independently of model variance. That is a scoping choice, not a claim that language models have no role: an LLM can usefully explain code, propose hypotheses, generate patch alternatives, or perform adversarial review, while structural reachability and compiler checks act as deterministic evidence gates around it. The result reported here comes from the rule-based path and is not a benchmark for any live model.

7 Discussion and Limitations

A clean build after patching establishes that a generated patch is syntactically valid and compatible with the real project’s dependency graph. It does not establish that the patch is correct at runtime. Confirming that the vulnerability is actually closed would require running the real free5GC UDR service and sending it a live HTTP request, which

Evidence boundary		
Static analysis	102 → 4	Demonstrated
Patch generation	4 patches	Demonstrated
Real upstream build	go build ./...	Demonstrated
Runtime closure	Live UDR request	Not demonstrated

Figure 7: Evidence boundary of the experiment: what is demonstrated and what remains unverified.

in turn requires the rest of the deployment this network function depends on: a MongoDB backend for its data store and registration with the free5GC NRF for service discovery. Standing up that deployment was outside the scope of this case study and is the most direct next step for this work (Figure 7).

Scope of generalisation. This is a single-file, single-family case study. It evaluates one real free5GC component, one disclosed vulnerability family (two CVEs), and one missing-`return` control-flow pattern in Go. The 96.1% reduction observed for this file and this set of HTTP entry points illustrates the effect of reachability filtering, but the figure will vary with project, language, architecture, and vulnerability type. Likewise, a patch that compiles is compatible with the tested upstream dependency graph, not necessarily correct in all cases. Broader conclusions would require several repositories, vulnerability classes, languages, and runtime security tests under reproducible conditions.

Partial-patch limitation. The most concrete limitation is the list-GET handler’s two-insertion fix: the current rule captures only the first match per function, and compilation does not reveal the incompleteness. The natural improvement is to have the patch interface emit all safe matches within a function, then re-run the upstream build and, ideally, a runtime regression test.

Threats to validity. Internal validity is supported by using real upstream source, a fixed set of entry points, a real grammar-based parser, and a full-module build. Reachability is a property of the static call graph and does not by itself prove a function is reachable in every runtime configuration. External validity is limited by the single case. The study is reproducible — the source is vendored, the commit hashes recorded, and the verification commands captured — but results may be influenced by environmental differences such as Go toolchain version, module availability, and network access.

8 Related Work

Our reachability filtering directly follows OpenAnt [1], which combines reachability-based code decomposition with LLM-driven adversarial verification and sandboxed dynamic testing

Table 5: Positioning against related security-automation work.

Work	Primary focus	Relation to this study
OpenAnt [1]	Reachability + LLM analysis	Methodological basis
AIxCC [3]	Autonomous cyber reasoning	Broader context
FuzzingBrain [5]	LLM-guided discovery+patch	Same pipeline shape
OSS-CRS [4]	Real OSS workloads	Real-target motivation
This work	Reachability + patch + build	Focused free5GC case

to discover previously unknown vulnerabilities across projects including OpenSSL, WordPress, and Flowise, reporting up to 97% reduction in analysis scope. We apply the same decomposition step to a single, real, disclosed CVE family rather than searching for unknown bugs, and add a verification step OpenAnt’s own evaluation does not report: confirming a generated patch against a live clone of the real upstream repository.

DARPA’s AI Cyber Challenge (AIxCC) produced seven open-sourced cyber-reasoning systems combining fuzzing, static analysis, and LLM agents for autonomous vulnerability discovery and patching [3]. FuzzingBrain, a fourth-place AIxCC finalist, autonomously discovered 28 vulnerabilities (including six zero-days) in real C and Java projects and patched 14, using LLMs as semantic navigators over reachability-partitioned code before dynamic verification [5]; its *analyse* → *direction* → *verify* shape mirrors this study’s reachability → triage → build-verification pipeline, independently arrived at by a funded finalist team. OSS-CRS extends this line of work by porting AIxCC-era systems to real OSS-Fuzz projects under resource budgets [4]. Our work is narrower than all of these: we do not attempt discovery of unknown vulnerabilities, only reachability-filtered triage and patching of already disclosed ones. It shares their emphasis on evaluating against real, not synthetic, targets (Table 5).

Difference from synthetic validation. This study should not be read as a replacement for dynamic testing. Static reachability narrows the code to inspect, and compiler verification catches integration errors, but neither replaces a real security test. The strongest future evaluation would connect the tested static pipeline to a real free5GC deployment and replay the request that demonstrated the vulnerability before the patch against the patched endpoint.

9 Reproducibility

The `src/reachability/` module and its documentation are hosted in the project repository for the free5GC case [9]. Both vulnerable and patched versions of the source are retained as case-study inputs, with the pre-fix and fix commit hashes recorded. The static procedure runs from the vendored case-study source; upstream verification clones the real module and runs the full Go build. Table 6 lists the minimum record needed to reproduce each reported number.

Table 6: Minimum reproducibility record.

Artifact	Purpose	Recorded value
Source commit	Fix provenance	Git hash
Toolchain	Reproduce parse/build	Versions
Entry points	Reachability boundary	4 handler names
Analysis result	Filtering measure	102 / 4 / 96.1%
Patch result	Generated edits	4 generated; 1 partial
Build result	Integration check	Baseline OK; patched OK

10 Future Work

First, runtime verification against a real free5GC UDR service: deploy the UDR with its MongoDB backend and NRF registration, replay the vulnerable request against the pre-fix revision, apply the generated patch, and replay again. A demonstrated negative that also preserves legitimate subscription behaviour would be far stronger evidence than compilation alone. Second, extend the patch generator to emit multiple edits per function: the present restriction to the first matching conditional block is exactly what leaves the list-GET handler partially corrected, and emitting all safe matches, then checking for conflicts and rebuilding, is the direct fix. Third, evaluate additional free5GC components and vulnerability classes to test whether the reduction and patch-success observations hold beyond this one file and family.

11 Conclusion

We presented a focused case study on a publicly disclosed vulnerability family in free5GC, showing that reachability-filtered static analysis and rule-based patch generation can be applied to a real vulnerability in an actual project’s upstream repository. The analysis narrowed 102 real functions to the 4 reachable handlers that matter — a 96.1% reduction — classified all four as CWE-285 matching the disclosed class, and produced patches that, applied to the real upstream module, preserve a clean `go build ./...` against the project’s actual dependency graph. We report this alongside its real limits: one handler is only partially corrected because the rule emits one match per function, and a successful build is not runtime confirmation. The contribution is not a claim of autonomous or complete remediation; it is a reproducible, real-target demonstration that reachability filtering focuses analysis and that upstream build verification provides more integration evidence than single-file compilation. The method extends naturally toward runtime security verification and a larger corpus of network-function software.

References

- [1] Nahum Korda and Gadi Evron. 2026. OpenAnt: LLM-Powered Vulnerability Discovery Through Code Decomposition, Adversarial Verification, and Dynamic Testing. *arXiv preprint arXiv:2606.19149* (2026). <https://arxiv.org/abs/2606.19149>
- [2] free5GC Project. <https://free5gc.org/>.
- [3] DARPA. 2025. AI Cyber Challenge marks pivotal inflection point for cyber defense. (August 2025). <https://www.darpa.mil/news/2025/aixcc-results>
- [4] Andrew Chin, Dongkwan Kim, Yu-Fu Fu, Fabian Fleischer, Youngjoon Kim, HyungSeok Han, Cen Zhang, Brian Junekyu Lee, Hanqing Zhao, and Taesoo Kim. 2026. OSS-CRS: Liberating AIxCC Cyber Reasoning Systems for Real-World Open-Source Security. *arXiv preprint arXiv:2603.08566* (2026). <https://arxiv.org/abs/2603.08566>
- [5] Ze Sheng, Qingxiao Xu, Jianwei Huang, Matthew Woodcock, Heqing Huang, Alastair F. Donaldson, Guofei Gu, and Jeff Huang. 2025. All You Need Is A Fuzzing Brain: An LLM-Powered System for Automated Vulnerability Detection and Patching. *arXiv preprint arXiv:2509.07225* (2025). <https://arxiv.org/abs/2509.07225>
- [6] Alexandr Kuznetsov, Yehor Yeromin, Oleksiy Shapoval, Kyryl Chernov, Maryna Popova, and Konstantin Serdukov. 2019. Automated Software Vulnerability Testing Using Deep Learning Methods. In *2019 IEEE 2nd Ukraine Conference on Electrical and Computer Engineering (UKRCON)*. IEEE, 837–841. <https://doi.org/10.1109/UKRCON.2019.8879997>
- [7] Ibrahim Almazyad, Safwan Elmadani, and Salim Hariri. 2024. A 5G and Beyond Testbed for Cybersecurity Research and Education. In *2024 IEEE/ACS 21st International Conference on Computer Systems and Applications (AICCSA)*. IEEE, 1–6.
- [8] free5GC/udr. CVE-2026-40248 / CVE-2026-40246 fix, upstream commit 86686276a7e226183ee786e3dd6714ec56c78fda, 2026.
- [9] Jothilingam Dheeraj. 2026. Final-Year-Project (src/reachability/). GitHub repository. <https://github.com/J-Dheeraj/Final-Year-Project>